Exact versus parallel approximate counting equivalence forces a zero-error second-level collapse

Simon C. Marshall, Scott Aaronson, Vedran Dunjko

doi:10.4230/LIPIcs.CCC.2025.5; CC BY-SA 4.0

Editorial note. Full native author source redistributed under its actual CC BY-SA4.0 license, not relabeled CC BY. The core main theorem native1 corresponds to official2; full author Sections2-3 retained with all support and natural interpretation proof. No mathematical claims corrected. Section4 explicitly declines formal reproduction of the sampling adaptations and is not counted as another complete main result.

Editorial note. Revision r002 inserts only three additional mathematical line breaks in the phi-prime multline in a separate render copy; every original symbol and term is preserved. Native original unchanged.

Allowed standard prerequisites (editorial).

- Random self-reducibility and checkability of counting classes; Toda theorem. Author cited statements of Feigenbaum-Fortnow, Arvind-Kobler and Toda retained verbatim in native Sections2-3; no recursive reproof of those standard prerequisites

1 Main theorem

In this section we prove our main theorem:

Theorem 1. *If* $\mathsf{P}^{\#\mathsf{P}} = \mathsf{BPP}_{\|}^{\mathsf{NP}}$ *then* $\mathsf{P}^{\#\mathsf{P}} = \mathsf{ZPP}^{\mathsf{NP}}$.

We will assume readers are broadly familiar with classes such as $\#\mathsf{P}$, where classes are undefined we use the definitions given in the complexity zoo [4]. We use the notation $\mathsf{A}_{\|}^{\mathsf{B}}$ for A with one set of parallel oracle calls to B.

It is useful to first give an informal description of the proof of theorem 1 before we proceed to the formal proof to give the reader a better idea of the purpose of each lemma. Broadly the proof relies on the notion of random-reducibility, where a given instance of a problem can be solved by a polynomial time algorithm with randomly selected instances of some other problem. If a problem can be randomly reduced to itself we say it is random self-reducible. The first key realisation for Theorem 1 is that a random self-reducible language that is in $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ is itself random reducible to NP, because randomly selecting x for a language in $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ gives rise to random set of non-adaptive NP

calls, which meets the definition of random reducibility to NP. Next, as #P is random self-reducible [9], the antecedent of Theorem 1 implies that it is random reducible to NP. The second key realisation is to notice that the proof by Feigenbaum and Fortnow that NP cannot be random self-reducible unless it is also in coNP/poly extends beyond self reducibility; any language that is random reducible to NP is in coNP/poly. By repeating the argument for coNP we show $\#\mathsf{P} \subseteq \mathsf{NP/poly} \cap \mathsf{coNP/poly}$. Arvind and Köbler [6] showed any checkable language in $\mathsf{NP/poly} \cap \mathsf{coNP/poly}$ is low for Σ_2^{P} which gives us a polynomial hierarchy collapse to Σ_2^{P}. The final collapse to $\mathsf{ZPP}^{\mathsf{NP}}$ comes from using the $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ algorithm to produce proofs verifiable in P^{NP}, thereby achieving zero error.

We begin by defining random reducibility. A problem is randomly self-reducible if any given instance of the problem can be solved probabilistically by a polynomial time algorithm with access to solved random instances of the same problem. For our purposes we will work with a similar concept: random-reducibility, which is the same except the random instances may be from a different problem. It should be stated that this is different from a 'randomized reduction', which is a randomised Karp reduction. Random reductions are sometimes called 1-locally random reductions to avoid this confusion.

Definition 2 (Random Reducible)**.** *A function f is **nonadaptively** $k(n)$ **random-reducible** to a function g if there are polynomial time computable functions σ and ϕ with the following two properties.*

(1) For all n and all $x \in \{0,1\}^n$ $f(x)$ can probably be solved by ϕ with instance of $g(y)$ for y selected by σ:

$$\Pr_r \left(f(x) = \phi(x, r, g(\sigma(1, x, r)), \ldots, g(\sigma(k, x, r))) \right) \geq 2/3$$

(2) For all n, all pairs $\{x_1, x_2\} \subset \{0,1\}^n$ and all i, if r is chosen uniformly at random then $\sigma(i, x_1, r)$ and $\sigma(i, x_2, r)$ are identically distrubuted.

If both conditions hold we say (σ, ϕ) is a random-reduction from f to g.

As we are dealing with no other forms of random reducibility we will just say 'f is rr to g' when f is non-adaptively $k(n)$ random-reducible to g for some polynomial k. If a function is rr to itself then we say the function is random self-reducible (rsr). A set is rr to another if its characteristic function is rr. A set of languages, A is rr to B if every $L \in \mathsf{A}$ is rr to some $L' \in \mathsf{B}$.

As with probabilistic classes like BPP, we can boost random reducibility an arbitrarily low (2^{-n}) failure probability.

Lemma 3 ([9])**.** *If function f is non-adaptively $k(n)$ random-reducible to g then for all $t(n)$, f is non-adaptively $24t(n)k(n)$ random-reducible to g where condition (1) holds for at least $1 - 2^{-t(n)}$ of the r's in $\{0,1\}^{24t(n)k(n)}$.*

Remark. *In general, this boosting may not work for a definition of random reducibility dealing with relations instead of functions as they may have multiple correct outputs. However, counting problems like* Perm*, which has only one correct output for a given input, can be boosted.*

Lemma 4 ([9])**.** *Any #P-complete language is rsr.*

This concludes our definitions, we can now state an intermediate theorem to our final result.

Theorem 5. *If* $\mathsf{P}^{\#\mathsf{P}} = \mathsf{BPP}_{\|}^{\mathsf{NP}}$ *then* $\mathsf{P}^{\#\mathsf{P}}$ *is random reducible to* NP.

Before we prove this, we will provide a number of smaller results. In the following lemma the notation $\mathsf{A}^{(\mathsf{B}[1])}$ means A with one oracle call to B.

Lemma 6. *Any language in* $\mathsf{P}^{\#\mathsf{P}[1]}$ *is random reducible to* $\#\mathsf{P}$.

Proof. As $\#\mathsf{P}$-complete languages are random self-reducible and only one oracle call is needed we can use a random reduction of $\#\mathsf{P}$ to give the answer to the oracle call and use the polynomial time ϕ algorithm to perform the rest of the P algorithm. □

The next lemma captures the intuition that $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ algorithms are 'almost' rr to NP, only missing the random element selection part of the definition (property (2)).

Lemma 7. *For all* L *in* $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ *there exists polynomial time computable functions* σ_B, ϕ_B *such that*

$$\Pr_r\left(L(x) = \phi_B(x, r, \mathsf{SAT}(\sigma_B(1, x, r)), \ldots, \mathsf{SAT}(\sigma_B(m, x, r)))\right) \geq 1 - 2^{-n}.$$

Proof. If $L \in \mathsf{BPP}_{\|}^{\mathsf{NP}}$ then there exists a polynomial time algorithm, A, to decide $x \in L$ which calls only one set of up to m non-adaptive NP queries. We assume the NP calls SAT for simplicity. Let the algorithm for $\sigma_B(i, x, r)$ run A until the step involving oracle queries and then output just the i'th query (assuming some arbitrary query ordering). The algorithm for ϕ_B is now just the algorithm A using the oracle calls produced asked by $\sigma_B(i, x, r)$ in place of directly making its own calls. Since both ϕ_B and σ_{B} have access to the same random string r they will both select the same oracle calls. □

Lemma 7 is useful as it proves that $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ fulfils property (1) of the definition of random reducibility to NP, but does not prove the random distribution of $\sigma_B(1, x, r)$. The next theorem shows that if a langauge already randomly reduces to $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ then this random reduction condition (condition (2)) is also fufilled, and thus the language is random reducible to NP

Lemma 8. *All languages random reducible to a language in* $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ *are random reducible to* NP.

Proof. Let L be a language which is random reducible to $L_B \in \mathsf{BPP}_{\|}^{\mathsf{NP}}$. We will demonstrate that L is rr to NP by writing out the definition of random reducibility to L_B then using lemma 7 to substitute the calls to L_B with a formula using calls to SAT. We can then rearange this into a format which directly makes calls to SAT and we show that these calls inherit the randomness property from the random reduction of L to L_B.

Let us assume the random reduction of L to L_B on input x uses k calls, from the definition of random reducibility there exists ϕ and σ such that

$$\Pr_r\left(L(x) = \phi(x, r, L_{\mathsf{BPP}}(\sigma(1, x, r)), \ldots, L_B(\sigma(k, x, r)))\right) \geq 1 - 2^{-n}.$$

Define $y_i := \sigma(i, x, r)$.

$$\Pr_r \left(L(x) = \phi(x, r, L_{\mathsf{B}}(y_1), \ldots, L_{\mathsf{BPP}}(y_k))\right) \geq 1 - 2^{-n}$$

Using lemma 7 we can substitute L_B with ϕ_B, σ_B and a random string r_i for the i'th call to L_B. In the following we assume each lemma-7-reduction uses m SAT calls.

$$\Pr_{r,r_1,\ldots r_k} \big(L(x) = \phi(x, r, \phi_B(y_1, r_1, \mathsf{SAT}(\sigma_B(1, y_1, r_1)), \ldots, \mathsf{SAT}(\sigma_B(m, y_1, r_1))),$$
$$\ldots, \phi_B(y_k, r_k, \mathsf{SAT}(\sigma_B(1, y_k, r_k)), \ldots, \mathsf{SAT}(\sigma_B(m, y_k, r_k)))\big) \geq 1 - (k+1)2^{-n}.$$

The failure probability $1 - (k+1)2^{-n}$ comes the failure probability of the k reductions combined with the failure probability from the random reduction.

We can now begin to combine elements of the reduction from L to L_B with reduction from L_B to SAT. We start with ϕ which we combine with ϕ_B to define ϕ', informally we can think of this as doing the two polynomial-time algorithms as a larger polynomial time algorithm.

$$\phi'(x, r, r_1, \ldots, r_k,$$
$$\mathsf{SAT}(\sigma_{\mathsf{B}}(1, \mathsf{y_1}, \mathsf{r_1})), \ldots \mathsf{SAT}(\sigma_{\mathsf{B}}(\mathsf{m}, \mathsf{y_k}, \mathsf{r_k}))) :=$$
$$\phi(x, r, \phi_B(y_1, r_1, \mathsf{SAT}(\sigma_B(1, y_1, r_1)), \ldots, \mathsf{SAT}(\sigma_B(m, y_1, r_1))), \ldots,$$
$$\phi_B(y_k, r_k, \mathsf{SAT}(\sigma_B(1, y_k, r_k)),$$
$$\ldots, \mathsf{SAT}(\sigma_B(m, y_k, r_k))$$

We use '$x\#y$' to denote y appended to x. Define $\mathbf{r} := r\#r_1\#\ldots\#r_k$.

Again we define a new function, σ', as the combination of two existing function, σ and σ_B:

$$\sigma'(i, j, x, \mathbf{r}) := \sigma_B(i, \sigma(j, x, r), r_j).$$

Which gives the following simplified equation:

$$\Pr_{\mathbf{r}} \left(L(x) = \phi'(x, \mathbf{r}, \mathsf{SAT}(\sigma'(1, 1, x, \mathbf{r})), \ldots \mathsf{SAT}(\sigma'(m, k, x, \mathbf{r}))\right) \geq 1 - (k+1)2^{-n}. \tag{1}$$

We will now directly check the definition of σ', ϕ' and the equation 1 against the definition of random-reducbile, recall the two conditions definition 2:

(1) For all n and all $x \in \{0,1\}^n$ $f(x)$ can probably be solved by ϕ with instance of $g(y)$ for y selected by σ:

$$\Pr_r \left(f(x) = \phi(x, r, g(\sigma(1, x, r)), \ldots, g(\sigma(k, x, r)))\right) \geq 2/3$$

(2) For all n, all pairs $\{x_1, x_2\} \subset \{0,1\}^n$ and all i, if r is chosen uniformly at random then $\sigma(i, x_1, r)$ and $\sigma(i, x_2, r)$ are identically distrubuted.

We can see that these two conditions are met:

(1) For sufficiently large n, $1 - (k+1)2^{-n} > 2/3$ therefore equation 1 is of the form:

$$\Pr_r \left(f(x) = \phi(x, r, \mathsf{SAT}(\sigma(1, x, r)), \ldots, \mathsf{SAT}(\sigma(k, x, r)))\right) \geq 2/3.$$

(2) For all n, all pairs $\{x_1, x_2\} \subset \{0,1\}^n$, all i and j, if r is chosen uniformly at random then $\sigma(j, x_1, r)$ and $\sigma(j, x_2, r)$ are identically distributed (as per their definition), therefore $\sigma'(i\#j, x_{1/2}, \mathbf{r})$ is identically distributed too. This is because the only dependence on x is through the identically distributed σ: $\sigma'(i\#j, x, \mathbf{r}) = \sigma_{\mathsf{B}}(i, \sigma(j, x, r), r_j)$. This holds for all r_i, r_j.

Thus fufilling the conditions for L to be random reducible to NP □

Finally, we can proceed with the proof of theorem 5

Proof of theorem 5. Toda [15] showed that $\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}[1]}$. Therefore, by the assumption of the theorem if $\mathsf{P}^{\#\mathsf{P}} = \mathsf{BPP}_{\|}^{\mathsf{NP}} = \mathsf{PH}$ then $\mathsf{P}^{\#\mathsf{P}} = \mathsf{P}^{\#\mathsf{P}[1]}$.

By lemma 6 we know $\mathsf{P}^{\#\mathsf{P}}$ is rr to $\#\mathsf{P}$. Under the assumption of this theorem, $\#\mathsf{P} \subseteq \mathsf{BPP}_{\|}^{\mathsf{NP}}$, $\mathsf{P}^{\#\mathsf{P}}$ is rr to $\mathsf{BPP}_{\|}^{\mathsf{NP}}$. By lemma 8 this means $\mathsf{P}^{\#\mathsf{P}}$ is rr to NP. □

The next result is heavily based on the work of Feigenbaum and Fortnow [9]. They show that if NP is random self reducible then coNP is in $\mathsf{NP/poly}$, while the result we are trying to show is slightly different than this, the method is very similar.

Theorem 9. *Any language that is random-reducible to a language in* NP *is also in* $\mathsf{coNP/poly} \cap \mathsf{NP/poly}$.

Proof. Let $\mathsf{AM}^{\mathsf{poly}}$ be AM with polynomial advice given to Arthur[1]. It turns out that $\mathsf{AM}^{\mathsf{poly}} = \mathsf{NP/poly}$ and $\mathsf{coAM}^{\mathsf{poly}} = \mathsf{coNP/poly}$ [9]. We will prove L is in $\mathsf{AM}^{\mathsf{poly}} \cap \mathsf{coAM}^{\mathsf{poly}}$, beginning with $L \in \mathsf{AM}^{\mathsf{poly}}$.

Suppose $L(x)$ is non-adaptively $k(n)$ random-reducible to $\mathsf{SAT}(x)$ with error probability 2^{-n}. We will adopt the notation $y_i = \sigma(i, x, r)$. For instance size n we will give Arthur the advice $(p_1, \ldots, p_k)$ where p_i is the probability that $y_i = 1$,

$$p_i = \Pr_r(\mathsf{SAT}(\sigma(i, x, r)) = 1).$$

As $\sigma(i, x, r)$ is distributed identically (given uniform random r) for all x this advice does not depend on the input x.

The proof system will consist of Arthur selecting $m = 9k^3$ random strings, $\{r_j\}_{j \in [m]}$, and passing this to Merlin, these random strings defines which SAT queries, $y_{0,j}, \ldots, y_{k,j}$, will be made. Merlin will hand back m 'transcripts'. These transcripts consist of a list $w_{i,j}$ which is either a witness for $y_{i,j}$ or is the string 'NIL' (which is interpreted as the claim that $y_{i,j} \notin \mathsf{SAT}$). For each of the m random strings we get the transcript:

$$Transcript(x, r_j) = (w_{0,j}, w_{1,j}, \ldots, w_{k,j})$$

Arthur performs three checks:

(1) For all i, j either the witness $w_{i,j}$ is 'NIL' or he checks the witness is valid for $y_{i,j} \in \mathsf{SAT}$

(2) Let $b_{i,j}$ be 0 if $w_{i,j}$ is 'NIL' and one otherwise. For all $j \in [m]$ Arthur checks that $\phi(x, r_j, b_{0,j}, \ldots, b_{k,j}) = 1$, i.e. If Merlin has told the truth then ϕ will accept.

[1]This is not the same as $\mathsf{AM/poly}$, as the latter must be an AM language for all advice, whereas the former only needs to be defined for the correct advice.

(3) For each $i \in [k]$ Arthur checks that more than $p_i m - 2\sqrt{km}$ of the $y_{i,j}$ have been proved to be in SAT, if this condition does not hold he rejects.

If these checks all pass, then Arthur accepts. The trick in this proof is this final condition. Over the random strings, each y_i has some probability of being in SAT with a given variance, by picking m to be large we force this variance to be small. With high probability, a correct answer lies in this range. Since Merlin cannot lie about yes instances (since he must prove them), he can only lie about no instances. However, if he lied too much it would be clear that not enough of y_i are in SAT, thus we could probabilistically guess he was cheating. To exploit this, m is picked precisely so Merlin cannot cheat on all j without exceeding his 'lying budget'. We will now formally show soundness and completeness.

Completness. If $x \in L$ and Merlin is honest, providing witnesses for all $y_{i,j}$ in SAT, condition (1) will always hold. Condition (2) holds with probability at least $1 - 2^{-n}$ (by the definition of rr). Therefore we just need to show condition (3) holds with high probability.

Let $Z_{i,j} := (y_{i,j} = 1)$ and $Z_i = \sum_{j=1}^{m} Z_{i,j}$. We defined our advice so $p_i = \mathbb{E}[Z_i]/m$, we can derive that $\mathrm{Var}(X) = p_i(1 - p_i)m < m$. We can use these properties and Chebyshev's inequality to bound our probability of failing check (2).

$$\begin{aligned}
&\Pr(Z_i \leq p_i m - 2\sqrt{km}) \\
&\leq \Pr(\|Z_i - p_i m\| \geq 2\sqrt{km}) \\
&= \Pr(\left\|Z_i - \bar{Z}_i\right\| \geq 2\sqrt{km}) \\
&\leq \mathrm{Var}(X)/4km \leq 1/(4k) \leq 1/4
\end{aligned}$$

The probability of failing any check given $x \in L$ is less than $\frac{1}{4} + 2^{-n}$ which is less than $1/3$ for large enough n. This proves completeness

Soundness. Suppose $x \notin L$, if Arthur accepts then all 3 checks must have passed for all $j \in [m]$. If some $y_{i,j}$ is in SAT but Merlin has provided $w_{i,j} = `NIL'$ then we say Merlin has lied about $y_{i,j}$, if check (1) passes Merlin has not lied about any 'yes' instances, so we disregard this case. The probability of the random reduction failing is 2^{-n} without any lies, so for all m reductions to fail Merlin must lie m times with probability $(1 - 2^{-n})^m > 1 - m2^{-n}$.

If Merlin lies m times there must be an i that he claims at least m/k of the $y_{i,j}$ are not in SAT when they are. To satisfy condition (3) at least $p_i m - 2\sqrt{km} + \mathbf{m/k}$ of the $y_{i,j}$ must be in SAT to leave 'room' for Merlin to lie m/k times without violating (3). The probability of this is given by a Chernoff bound on the random variable Z_i, as defined above.

$$\begin{aligned}
&\Pr\left(Z_i \geq p_i m - 2\sqrt{km} + m/k\right) \\
&= \Pr\left(Z_i - p_i m \geq m/k - 2\sqrt{km}\right) \\
&= \Pr\left(Z_i - p_i m \geq 9k^3/k - 2\sqrt{k9k^3}\right) \\
&= \Pr\left(Z_i - p_i m \geq 3k^2\right) \\
&\leq e^{-2\times 9k^4/9k^3} = e^{-2\times k} \\
&\leq 1/(4k)
\end{aligned}$$

Combining this with the normal probability that the random reduction fails even with correct oracle answers (which had a 2^{-n} probability) gives an acceptance probability given $x \notin L$ of less than $m2^{-n} + 1/(4k) < 1/3$ for large enough n. This proves soundness.

The previous analysis can just as easily be repeated for $\mathsf{coAM}^{\mathsf{poly}}$ with Merlin proving no instances, giving $L \in \mathsf{AM}^{\mathsf{poly}} \cap \mathsf{coAM}^{\mathsf{poly}}$. By Feigenbaum and Fortnow we know this equals $\mathsf{NP/poly} \cap \mathsf{coNP/poly}$. □

To prove the next Theorem we must recall a crucial result by Arvind and Köbler, which checkability [6, 4]. The notion of checkability is somewhat involved but intuitively has to do with whether efficent programs that decide the set can themselves be efficently checked. Since we will just need the checkability of PP and #P to deploy the following theorem we will not formally define it, instead we refer the reader to [6, 7].

Theorem ([6], Theorem 22)**.** *Checkable sets in* $\mathsf{NP/poly} \cap \mathsf{coNP/poly}$ *are low for* Σ_2^{P}.

This result is imperative to proceed, as while it is true that $\mathsf{NP} \subset (\mathsf{NP} \cap \mathsf{coNP})/\mathsf{poly}$ implies $\mathsf{PH} = \mathsf{ZPP}^{\mathsf{NP}}$, the same is not true for $\mathsf{NP} \subset (\mathsf{NP/poly}) \cap (\mathsf{coNP/poly})$.

Lemma 10. *If* $\mathsf{P}^{\#\mathsf{P}} \subset (\mathsf{coNP/poly} \cap \mathsf{NP/poly})$ *then* $\mathsf{P}^{\#\mathsf{P}} = \Sigma_2^{\mathsf{P}} \cap \Pi_2^{\mathsf{P}}$

Proof. By Toda $\mathsf{P}^{\#\mathsf{P}} = \mathsf{P}^{\mathsf{PP}}$ [15]. If P^{PP} is in $\mathsf{coNP/poly} \cap \mathsf{NP/poly}$ then clearly so is PP, combining this with the existence of PP-complete checkable sets [6] we know that PP is low for Σ_2^{P}.

This lowness implies we can put $\mathsf{P}^{\#\mathsf{P}}$ in Σ_2^{P} by the following series of inclusions

$$\mathsf{P}^{\#\mathsf{P}} = \mathsf{P}^{\mathsf{PP}} \subseteq (\Sigma_2^{\mathsf{P}})^{\mathsf{PP}} = \Sigma_2^{\mathsf{P}}.$$

As $\Sigma_2^{\mathsf{P}} \subseteq \mathsf{P}^{\#\mathsf{P}}$ by Toda [15] we can fix the previous inclusion into an equality: $\Sigma_2^{\mathsf{P}} = \mathsf{P}^{\#\mathsf{P}}$.

As $\Pi_2^{\mathsf{P}} \subseteq \mathsf{P}^{\#\mathsf{P}}$ (Toda [15]) we get $\Pi_2^{\mathsf{P}} \subseteq \Sigma_2^{\mathsf{P}}$. Which gives us the final equality:

$$\mathsf{P}^{\#\mathsf{P}} \subseteq \Sigma_2^{\mathsf{P}} \subseteq \mathsf{P}^{\Pi_2^{\mathsf{P}}} = \mathsf{P}^{\Sigma_2^{\mathsf{P}} \cap \Pi_2^{\mathsf{P}}} = \Sigma_2^{\mathsf{P}} \cap \Pi_2^{\mathsf{P}}$$

□

The previous lemma has achieved a collapse to the second level but we can collapse to $\mathsf{ZPP}^{\mathsf{NP}}$ by using the proveability of languages in $\Sigma_2^{\mathsf{P}} \cap \Pi_2^{\mathsf{P}}$ to make the $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ algorithm zero-error.

Lemma 11. *All languages in* $\mathsf{BPP}^{\mathsf{NP}} \cap \Sigma_2^{\mathsf{P}} \cap \Pi_2^{\mathsf{P}}$ *are in* $\mathsf{ZPP}^{\mathsf{NP}}$

Proof. Fix some $L \in \mathsf{BPP}^{\mathsf{NP}} \cap \Sigma_2^{\mathsf{P}} \cap \Pi_2^{\mathsf{P}}$. We can check if $x \in L$ using the $\mathsf{BPP}^{\mathsf{NP}}$ algorithm. Use the $\mathsf{BPP}^{\mathsf{NP}}$ program to determine a proof of this fact for either the Σ_2^{P} verifier or the Π_2^{P} verifier. We can test either proof in P^{NP} using the verifiers from either Σ_2^{P} or Π_2^{P}. With probability $1 - 2^{-n}$ the $\mathsf{BPP}^{\mathsf{NP}}$ algorithm succeeds in producing a valid proof in polynomial time, any valid proof proves or disproves $x \in L$. Therefore in polynomial time the algorithm has successfully determined if $x \in L$ with probability $1 - 2^{-n}$ and never incorrectly answers, placing $L \in \mathsf{ZPP}^{\mathsf{NP}}$. □

This completes the proof of our main theorem which is given below.

Theorem. *If* $\mathsf{P}^{\#\mathsf{P}} = \mathsf{BPP}_{\|}^{\mathsf{NP}}$ *then* $\mathsf{P}^{\#\mathsf{P}} = \mathsf{ZPP}^{\mathsf{NP}}$

Summary of proof. If $\mathsf{P}^{\#\mathsf{P}} = \mathsf{BPP}_{\|}^{\mathsf{NP}}$ then $\mathsf{P}^{\#\mathsf{P}}$ is random reducible to NP (Theorem 4).

If $\mathsf{P}^{\#\mathsf{P}}$ is random reducible to NP then $\mathsf{P}^{\#\mathsf{P}}$ is in $\mathsf{coNP/poly} \cap \mathsf{NP/poly}$ (Theorem 9).

If $\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{coNP/poly} \cap \mathsf{NP/poly}$ then $\mathsf{P}^{\#\mathsf{P}} = \Sigma_2^{\mathsf{P}} \cap \Pi_2^{\mathsf{P}}$ (Lemma 10).

If $\mathsf{P}^{\#\mathsf{P}} = \Sigma_2^{\mathsf{P}} \cap \Pi_2^{\mathsf{P}}$ and $\mathsf{P}^{\#\mathsf{P}} = \mathsf{BPP}^{\mathsf{NP}}$ then $\mathsf{P}^{\#\mathsf{P}} = \mathsf{ZPP}^{\mathsf{NP}}$ (Lemma 11). □

Remark. *Interestingly we can perform all of the oracle calls required to produce the proof of $x \in L$ in one parallel step, it then only takes one extra Oracle call to check this proof. Thus it is possible to answer correctly with probability $1 - 2^{-n}$ or with 'I don't know' after only 2 rounds of parallel* NP *oracle calls.*

Remark. *Feigenbaum and Fortnow [9] showed their proof applies beyond non-adaptive random self-reducibility, up to logarithmically many adaptive steps are allowed. For the same reasons, our proof could be extended to logarithmically many rounds of polynomially many parallel oracle calls.*

The following theorem formalises the 'natural interpretation' of our result given in the intro.

Theorem 12 (Natural interpretation)**.** *If* $\mathsf{P}_{\|}^{\mathsf{ExactCount}} = \mathsf{P}_{\|}^{\mathsf{ApproxCount}}$ *then* $\mathsf{P}^{\#\mathsf{P}} = \mathsf{ZPP}^{\mathsf{NP}}$

This result becomes easy to show after Theorem 15, so we will prove it at the end of the next section.

2 PostBQP and PostBPP

An immediate consequence of the last section's main theorem is to the question of $\mathsf{PostBQP} \stackrel{?}{=} \mathsf{PostBPP}$, i.e. is a poly-time quantum computer with postselection equal to a classical computer with postselection? The best existing answer was given when Aaronson showed $\mathsf{PostBQP} = \mathsf{PP}$ [2], thus equality would imply $\mathsf{PH} = \mathsf{BPP}^{\mathsf{NP}}$ (a collapse to the third level). However, as $\mathsf{PostBPP} \subseteq \mathsf{BPP}_{\|}^{\mathsf{NP}}$ [13, 11] our theorem can be used to improve this collapse to the second level. Before we provide proof of this we formally define $\mathsf{PostBQP}$ and the operator $\mathsf{BP}\cdot$ which makes a complexity class probabilistic.

Definition 13. $\mathsf{PostBQP}$ *is the set of languages for which there exists a uniform family of polynomial width and polynomial depth quantum circuits $\{C_n\}_{n\geq 1}$ such that for any input x,*

(i) There is a non-zero probability of measuring the first qubit of $C_n |0\ldots0\rangle |x\rangle$ to be $|1\rangle$.

(ii) If $x \in L$, conditioned on the first qubit being $|1\rangle$, the second qubit is $|1\rangle$ with probability at least 2/3.

(iii) If $x \notin L$, conditioned on the first qubit being $|1\rangle$, the second qubit is $|1\rangle$ with probability at most 1/3.

$\mathsf{PostBPP}$ can be defined similarly with classical circuits and additional input of random bits.

Definition 14. *Let K be a complexity class. A language L, is in $\mathsf{BP} \cdot \mathsf{K}$ if there exists a language $\mathsf{A} \in \mathsf{K}$ and a polynomial p such that for all strings x.*

$$Pr(r \in \{0,1\}^{p(|x|)} : x \in L \iff x\#r \in A) \geq 2/3$$

This covers the necessary definitions and we can proceed with the collapse result of interest.

Theorem 15. *If $\mathsf{PostBQP} = \mathsf{PostBPP}$ then $\mathsf{P}^{\#\mathsf{P}} = \mathsf{ZPP}^{\mathsf{NP}}$ and the polynomial hierarchy collapses to the second level.*

Proof. Assume $\mathsf{PostBQP} = \mathsf{PostBPP}$. By Toda $\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}}$, by Aaronson [1] $\mathsf{PostBQP} = \mathsf{PP}$ and by $\mathsf{PostBPP} \subseteq \mathsf{BPP}_{\|}^{\mathsf{NP}}$ [13, 11]. This gives:

$$\mathsf{PP} \subseteq \mathsf{BPP}_{\|}^{\mathsf{NP}} \text{ and } \mathsf{P}^{\#\mathsf{P}} = \mathsf{P}^{\mathsf{PP}} \subseteq \mathsf{P}^{\mathsf{BPP}^{\mathsf{NP}}} = \mathsf{BPP}^{\mathsf{NP}} = \mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}}.$$

Therefore $\mathsf{P}^{\#\mathsf{P}} = \mathsf{PH}$.

At this point we recall an interesting lemma from Toda [15] to continue.

$$\mathsf{PP}^{\mathsf{PH}} \subseteq \mathsf{BP} \cdot \mathsf{PP}$$

Clearly $\mathsf{PH} \subseteq \mathsf{PP}^{\mathsf{PH}}$ and it can be shown that if $\mathsf{PP} \subseteq \mathsf{BPP}_{\|}^{\mathsf{NP}}$ then $\mathsf{BP} \cdot \mathsf{PP} \subseteq \mathsf{BPP}_{\|}^{\mathsf{NP}}$ (a full proof follows in lemma 16), therefore

$$\mathsf{PH} \subseteq \mathsf{BP} \cdot \mathsf{PP} \subseteq \mathsf{BPP}_{\|}^{\mathsf{NP}}.$$

Since $\mathsf{PH} = \mathsf{P}^{\#\mathsf{P}}$ and $\mathsf{BPP}_{\|}^{\mathsf{NP}} \subseteq \mathsf{PH}$ we can conclude $\mathsf{P}^{\#\mathsf{P}} = \mathsf{BPP}_{\|}^{\mathsf{NP}}$. We now have the condition $\mathsf{P}^{\#\mathsf{P}} = \mathsf{BPP}_{\|}^{\mathsf{NP}}$ so by theorem 1 we get $\mathsf{P}^{\#\mathsf{P}} = \mathsf{ZPP}^{\mathsf{NP}}$, the final result. □

Lemma 16. *If PP is in $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ then so is $\mathsf{BP} \cdot \mathsf{PP}$*

Proof. Fix some $L \in \mathsf{BP} \cdot \mathsf{PP}$. By the definition, there exists $A \in \mathsf{PP}$ which defines L. Assume PP is in $\mathsf{BPP}_{\|}^{\mathsf{NP}}$, therefore A is in $\mathsf{BPP}_{\|}^{\mathsf{NP}}$. Therefore L is in $\mathsf{BP} \cdot \mathsf{BPP}_{\|}^{\mathsf{NP}}$.

Note that majority-vote amplification can be used on $\mathsf{BP} \cdot \mathsf{PP}$ and $\mathsf{BPP}_{\|}^{\mathsf{NP}}$ to decrease the probability of failure to less than $1/6$.

If we combine the random coin flips of both the $\mathsf{BP} \cdot \mathsf{PP}$ and BPP parts of the algorithm we get a single failure probability below $1/3$, which fits the definition of $\mathsf{BPP}_{\|}^{\mathsf{NP}}$. □

Theorem 15 provides a quick a simple proof of Theorem 12 (the natural interpretation of our result).

Proof of Theorem 12. By definition $\#\mathsf{P} = \mathsf{ExactCount}$. O'Donnell and Say show $\mathsf{P}_{\|}^{\mathsf{ApproxCount}} = \mathsf{PostBPP}$ [13]. Therefore we can rewrite the antecedent of the Theorem as $\mathsf{P}_{\|}^{\#\mathsf{P}} = \mathsf{PostBPP}$.

Any problem in PP can be solved with one $\#\mathsf{P}$ query, which can obviously be done in one parallel round of oracle calls, so $\mathsf{PP} \subseteq \mathsf{P}_{\|}^{\#\mathsf{P}}$. This gives $\mathsf{PP} = \mathsf{PostBPP}$ (as we already know $\mathsf{PostBPP} \subseteq \mathsf{PP}$ and by Theorem 15 $\mathsf{P}^{\#\mathsf{P}} = \mathsf{ZPP}^{\mathsf{NP}}$. □

References

[1] Scott Aaronson. Quantum computing, postselection, and probabilistic polynomial-time. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 461(2063):3473–3482, 2005.

[2] Scott Aaronson. The equivalence of sampling and searching. *Theory of Computing Systems*, 55(2):281–298, 2014.

[3] Scott Aaronson and Alex Arkhipov. The computational complexity of linear optics. In *Proceedings of the forty-third annual ACM symposium on Theory of computing*, pages 333–342, 2011.

[4] Scott Aaronson, Greg Kuperberg, and Christopher Granade. The complexity zoo, 2005.

[5] Dorit Aharonov, Xun Gao, Zeph Landau, Yunchao Liu, and Umesh Vazirani. A polynomial-time classical algorithm for noisy random circuit sampling. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 945–957, 2023.

[6] V. Arvind and Johannes Köbler. New lowness results for zppnp and other complexity classes. *Journal of Computer and System Sciences*, 65(2):257–277, 2002.

[7] Manuel Blum and Sampath Kannan. Designing programs that check their work. *Journal of the ACM (JACM)*, 42(1):269–291, 1995.

[8] Michael J Bremner, Ashley Montanaro, and Dan J Shepherd. Average-case complexity versus approximate simulation of commuting quantum computations. *Physical review letters*, 117(8):080501, 2016.

[9] J Feigenbaum and L Fortnow. On the random-self-reducibility of complete sets, university of chicago technical report 90-22. *Computer Science Department*, 20, 1990.

[10] Keisuke Fujii, Hirotada Kobayashi, Tomoyuki Morimae, Harumichi Nishimura, Shuhei Tamate, and Seiichiro Tani. Power of quantum computation with few clean qubits. *arXiv preprint arXiv:1509.07276*, 2015.

[11] Yenjo Han, Lane A Hemaspaandra, and Thomas Thierauf. Threshold computation and cryptographic security. *SIAM Journal on Computing*, 26(1):59–78, 1997.

[12] Tomoyuki Morimae. Hardness of classically sampling the one-clean-qubit model with constant total variation distance error. *Physical Review A*, 96(4):040302, 2017.

[13] Ryan O'Donnell and AC Cem Say. The weakness of ctc qubits and the power of approximate counting. *ACM Transactions on Computation Theory (TOCT)*, 10(2):1–22, 2018.

[14] Larry Stockmeyer. On approximation algorithms for# p. *SIAM Journal on Computing*, 14(4):849–861, 1985.

[15] Seinosuke Toda. Pp is as hard as the polynomial-time hierarchy. *SIAM Journal on Computing*, 20(5):865–877, 1991.

[16] Leslie G Valiant and Vijay V Vazirani. Np is as easy as detecting unique solutions. In *Proceedings of the seventeenth annual ACM symposium on Theory of computing*, pages 458–463, 1985.